Case Statement

➢ **Syntax**

```
case (expression)
   case_0:  begin
              statements;
            end
   case_1:  begin
              statements;
            end
   case_2:  begin
              statements;
            end
   default: begin
              statements;
            end
endcase
```

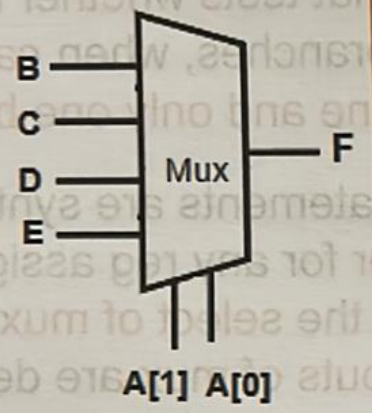
- The case/casex statement is a multi-way decision statement that tests whether an expression matches one of the branches, when case statement is executed, one and only one branch will be executed.
- The case statements are synthesized by generating a multiplexer for any reg assigned within the statements, the select of mux is the case expression and data inputs of mux are determined by the expressions on the right-hand sides of the assignments.
- case statement is used for mutually exclusive cases.
- case statements are **only** used inside always block.
- Default branch used to cover all the unspecified branches; it executes when none of the branches are matched.

© 2022 Eng. Ali Eltemsah

Multiplexer

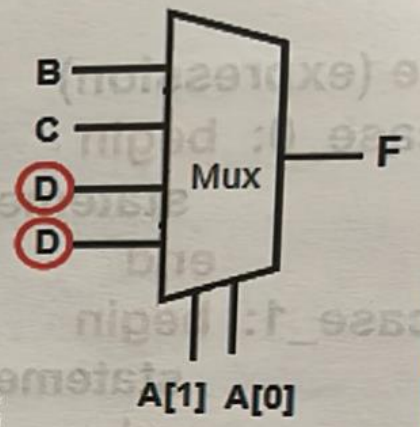
- **4X1 Multiplexer**

```
reg [1:0] F ;

always(*)
 begin
  case (A)
   2'b00 : F = B
   2'b01 : F = C ;
   2'b10 : F = D ;
   2'b11 : F = E ;
  endcase
 end
```



- **4X1 Multiplexer**

```
reg [1:0] F ;

always(*)
 begin
  casex (A)
   2'b00 : F = B ;
   2'b01 : F = C ;
   2'b1x : F = D ;
  endcase
 end
```



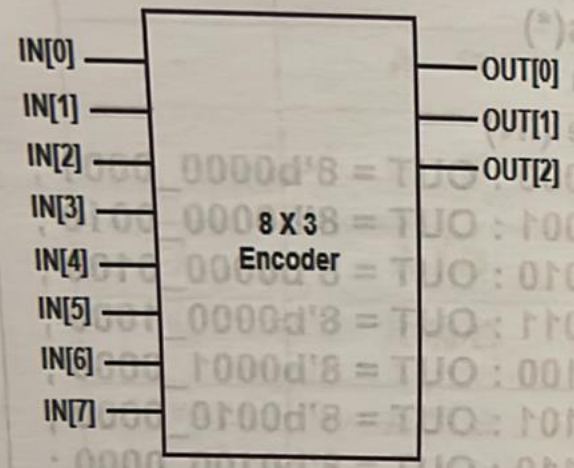
In general, it is not recommended to use 'x' in your code as not all simulators understand it

Note: We don't need to put default branch as all the possible cases are listed.

© 2021 Eng. Ali Mohamed

Encoder

Behavior ➡ Function



Truth Table

IN	OUT
0000_0001	000
0000_0010	001
0000_0100	010
0000_1000	011
0001_0000	100
0010_0000	101
0100_0000	110
1000_0000	111

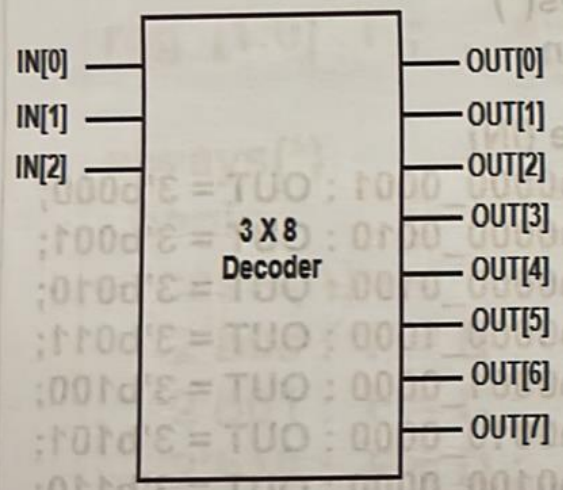
8X3 Encoder

```
always(*)
 begin

  case (IN)
   8'b0000_0001 : OUT = 3'b000;
   8'b0000_0010 : OUT = 3'b001;
   8'b0000_0100 : OUT = 3'b010;
   8'b0000_1000 : OUT = 3'b011;
   8'b0001_0000 : OUT = 3'b100;
   8'b0010_0000 : OUT = 3'b101;
   8'b0100_0000 : OUT = 3'b110;
   8'b1000_0000 : OUT = 3'b111;

   default : OUT = 3'b000;

  endcase
 end
```

© 2022 Eng. Ali Eltemsah

Decoder

Behavior ➡ Function



Truth Table

IN	OUT
000	0000_0001
001	0000_0010
010	0000_0100
011	0000_1000
100	0001_0000
100	0010_0000
101	0100_0000
110	1000_0000

3X8 Decoder

```
always(*)
 begin
   case (IN)
    3'b000 : OUT = 8'b0000_0001 ;
    3'b001 : OUT = 8'b0000_0010 ;
    3'b010 : OUT = 8'b0000_0100 ;
    3'b011 : OUT = 8'b0000_1000 ;
    3'b100 : OUT = 8'b0001_0000 ;
    3'b101 : OUT = 8'b0010_0000 ;
    3'b110 : OUT = 8'b0100_0000 ;
    3'b111 : OUT = 8'b1000_0000 ;
   endcase
 end
```

© 2022 Eng. Ali Eltemsah

Arithmetic Operators

Sign	Usage	Meaning	Comments
+	A + B	Add	Infer an n-bits adder/subtractor from synthesis tool designWare library
-	A - B	Subtract	
*	A * B	Multiply	Infer a large parallel Multiplier from synthesis tool designWare library (High Power)
/	A / B	Divide	Not synthesizable
%	A % B	Modulus	Not synthesizable
**	A ** B	A to the power B	Not synthesizable

- Usually Arithmetic operators like (+, -*, / , %) are used for the basic usage, but in real design, these operators are replaced by different Adding, Multiplication and Division algorithms.

© 2022 Eng. Ali Eltemsah

Bitwise Operators

- Operate on Vectors (bit-by-bit)
- Mismatched length operands are zero extended

Sign	Usage	Description	Sign	Usage	Description
&	A & B	AND each bit of A with B	~&	~ (A & B)	NAND each bit of A with B
|	A | B	OR each bit of A with B	~|	~ (A | B)	NOR each bit of A with B
^	A ^ B	XOR each bit of A with B	~^	(A ~^ B)	XNOR each bit of A with B
~	~A	Invert each bit of A			

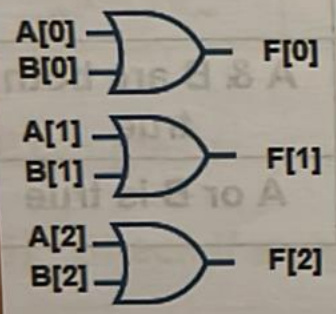
© 2022 Eng. Ali Eltemsah

Bitwise Operators

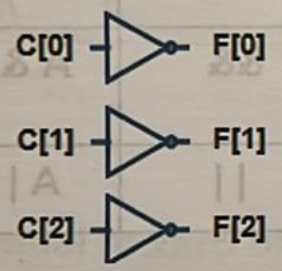
Examples:-

```
reg  [2:0]  A, B, C;
wire [2:0]  F ;
```

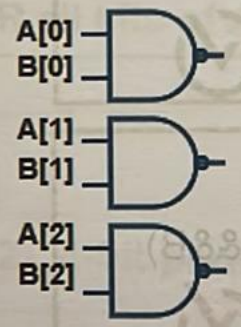
F = A | B



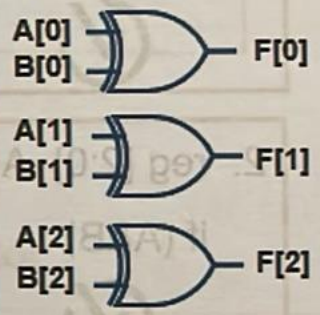
F = ~ C



F = ~ (A & B)



F = (A ^ B)



© 2022 Eng. Ali Eltemsah

Logical Operators

- Take **one-bit** inputs and produce a **1-bit** result
- Generally used in conditions of if-else statements
- Examples:-

1. reg A, B;

   if (A&&B) ✓ if (A&B) ✓

2. reg [2:0] A, B;

   if (A&B) ✓ if (A&&B) ✗

Sign	Usage	Description
!	!A	A is not true
&&	A && B	A & B are both true
||	A || B	A or B is true

© 2022 Eng. Ali Eltemsah

Unary Reduction Operators

Sign	Usage	Description
&	&A	AND all bits of A together
~&	~&A	NAND all bits of A together
|	|A	OR all bits of A together
~|	~|A	NOR all bits of A together
^	^A	XOR all bits of A together
~^ or ^~	~^A	XNOR all bits of A together

Examples :-

1) Check if a signal equal to zero or not

```
reg [2:0] A;
wire C;
assign C = ~|A ;
```

2) 4-bit parity generator

```
reg [3:0] data;
wire parity;
assign parity = ^data ;
```

© 2022 Eng. Ali Eltemsah

Equality & Relational Operators

Sign	Usage	Description
==	A == B	True if A is equal to B
!=	A != B	True if A is not equal to B
<	A < B	True if A is less than B
>	A > B	True if A is greater than B
<=	A <= B	True if A is less than or equal B
>=	A >= B	True if A is greater than or equal B

- Take **n-bits** inputs and produce a **1-bit** result
- Generally used in **conditions** of if-else statements
- Relational operators compare operands bit by bit and return 1 if expression is true, else return 0
- Equality & Relational Operators Infer a **comparator** from synthesis tool designWare library

© 2022 Eng. Ali Eltemsah

Shift Operators

Logical Shift

Sign	Usage	Meaning	Description
<<	A << B	Logical Shift Left	Shift A left by B bits, zero fill on right
>>	A >> B	Logical Shift Right	Shift A right by B bits, zero fill on left

Arithmetic Shift

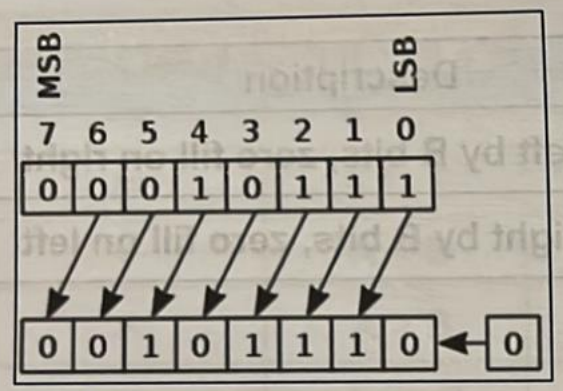
Sign	Usage	Meaning	Description
<<<	A <<< B	Arithmetic Shift Left	Shift A left by B bits, zero fill on right
>>>	A >>> B	Arithmetic Shift Right	Shift A right by B bits, sign fill on left if A is signed, zero filled if not

- **Shift left** is used for Multiplication by 2^n where n is the number of shifts.
- **Shift right** is used for division by 2^n where n is the number of shifts.
- **Logical shift** is used for the unsigned numbers
- **Arithmetic shift** is used for the signed number as it preserves the sign bit.

© 2022 Eng. Ali Eltemsah

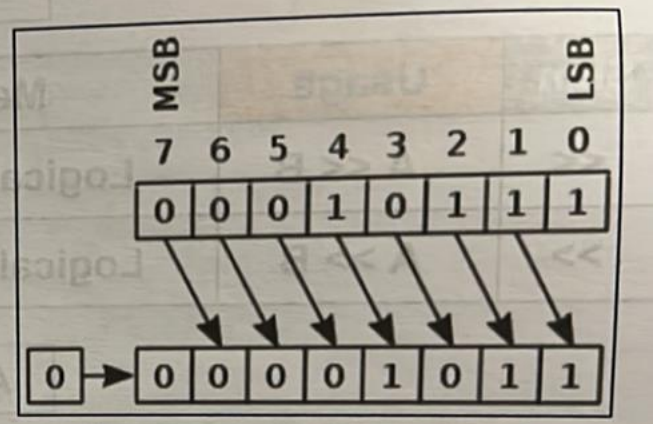
Shift Operators

Logical Shift



Shift left one bit

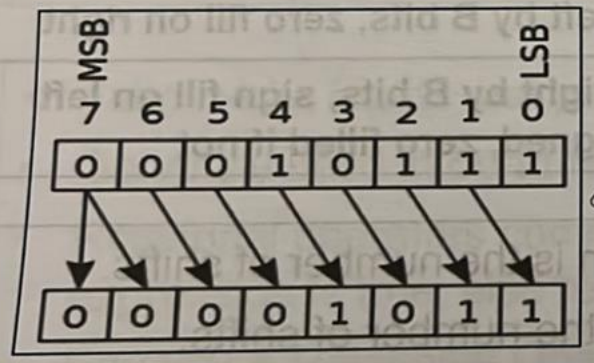
X<<1



Shift right one bit

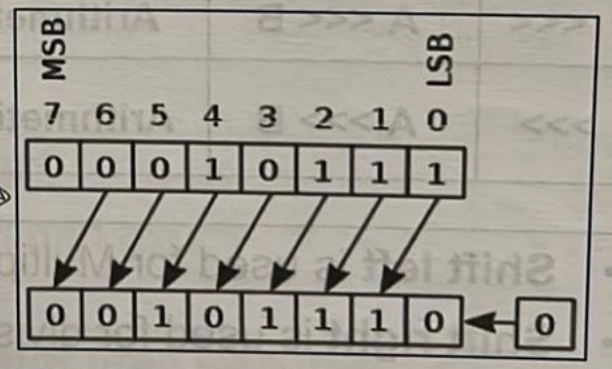
X<<1

Arithmetic Shift



Arithmetic Shift left one bit

X<<<1



Arithmetic Shift right one bit

X>>>1

© 2022 Eng. Ali Eltemsah

Miscellaneous Operators

Concatenation & Replication

Sign	Usage	Description
{}	{A , B}	Concatenate A and B, creating a longer vector
{{}}	{A {B}}	Replicate B A times, creating a longer vector

Example:-

```
wire [2:0]   A, B ;          //  if   A=3'b010;     B=3'b101;
wire [5:0]   C, D ;

assign C = { A, B }  ;       //  C = 010110

assign D = { A[1:0] {B} } ;  //  C = 101101
```

© 2022 Eng. Ali Eltemsah

Comments on Operations

1. Operators expanded to the largest element in the expression, before evaluation
   - o Unsigned vectors are zero filled
   - o Signed vectors are sign bit filled

2. {} and {{{} are done BEFORE expansion, and then treated as an operand at new width.

3. If Large variable assigned to a shorter variable, high order bits are lost.

4. In arithmetic operations, If any operand is unsigned, arithmetic is unsigned.

5. If ALL operators are signed, signed arithmetic is conducted.

- Example : wire [3:0] A, B, C;

  assign C = ({A,A} >> 2) | B[1:0]

  //C = lower 4 bits of (8 bits | 0000 00B[1]B[0])

© 2022 Eng. Ali Eltemsah

Verilog Timescale

- Verilog simulation depends on how time is defined because the simulator needs to know what #1 means in terms of time. The `timescale compiler directive specifies the **time unit** and **time precision** for the modules that follow it.

```
`timescale <time_unit>/<time_precision>
```

- Examples:-

1. `timescale 1ns/1ns
   - Time unit = 1 ns
   - Time precision = 1 ns

2. `timescale 1ns/100ps
   - Time unit = 1 ns
   - Time precision = 100 ps (0.1 ns)

Character	TimeUnit
ms	milliseconds
us	microseconds
ns	nanoseconds
ps	picoseconds
fs	Femtoseconds

© 2022 Eng. Ali Eltemsah

Verilog Timescale

`timescale 1ns/1ns

#0.49
> simulator round it down to **0**

#0.5
> simulator round it up to **1**

#0.3
> simulator round it down to **0**

#0.8
> simulator round it up to **1**

`timescale 1ns/100ps

#0.49
>simulator round it up to **0.5**

#0.5
> simulator don't round it **1**

#0.33
>simulator round it down to **0.3**

#0.88
> simulator round it up to **0.9**

`timescale 1ns/10ps

#0.49
> simulator doesn't round it

#0.5
> simulator doesn't round it

#0.3
> simulator doesn't round it

#0.8
> simulator doesn't round it

© 2022 Eng. Ali Eltemsah

Verilog System Functions

$display

- Syntax: $display (“format_string”, variable_1, variable_2, …)
- It is used to display or print messages or variables values to track the simulation flow from log files.
- You need to write the $display statement every time to track variable value.
- Example: $display (“Final Counter Value at time =%0t = 0x%0h“, $time, counter_tb) ;

$monitor

- Syntax: $monitor (“format_string”, variable_1, variable_2, …) ;
- It used to automatically print out variables values whenever one of variables in the argument list changes. It achieve the same effect of calling $display after every time any of its arguments get updated.
- You need to write the $monitor statement only once in our code.
- Example: $monitor ("time=%0t, Counter=0x%0h", $time, counter_tb)

© 2022 Eng. Ali Eltemsah

Verilog System Functions

➢ **Display Formats**

Character	Format
%h %H	Display in hexadecimal format
%d %D	Display in decimal format
%b %B	Display in binary format
%t, %T	Display in time format
%4b %4B	Display in binary format with 4-bits width
%2h %2H	Display in hexadecimal format with 2 hexadecimal characters (8-bits width)

© 2022 Eng. Ali Eltemsah